

Let us discuss memory layout in the C program:

🤔 C program memory layout in C mainly comprises six components: heap, stack, code segment, command-line arguments, and uninitialized and initialized data segments. Each of these segments has its read and write permissions. A segmentation fault occurs when a program tries to access any of the segments in a way that is not allowed, which is also a common reason for the program to crash.

👉 Text Segment

1. After we compile the program, a binary file is generated, which is used to execute our program by loading it into RAM. This binary file contains instructions, and these instructions get stored in the text segment of the memory.
2. Text segment has read-only permission that prevents the program from accidental modifications.
3. Text segment in RAM is shareable, so a single copy is required in the memory for frequent applications like a text editor, shells, etc.

🎯 Heap

Heap is used for memory which is allocated during the run time (dynamically allocated memory). Heap generally begins at the end of bss segment and, they grow and shrink in the opposite direction of the Stack. Commands like malloc, calloc, free, realloc, etc are used to manage allocations in the heap segment which internally use system calls to change memory allocation within the heap segment. A heap data segment is shared among modules loading dynamically and all the shared libraries in a process.



Stack

The stack segment follows the LIFO (Last In First Out) structure and grows down to the lower address, but it depends on computer architecture. Stack grows in the direction opposite to heap. Stack segment stores the value of local variables and values of parameters passed to a function along with some additional information like the instruction's return address, which is to be executed after a function call.



Uninitialized data segment

An uninitialized data segment is also known as bss (block started by symbol). The program loaded allocates memory for this segment when it loads. Every data in bss is initialized to arithmetic 0 and pointers to null pointer by the kernel before the C program executes. BSS also contains all the static and global variables, initialized with arithmetic 0. Because values of variables stored in bss can be changed, this data segment has read-write permissions.

